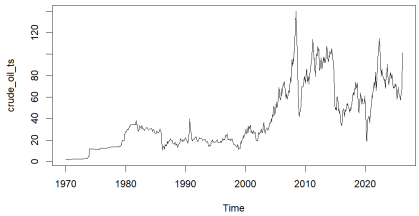
Forecasting Crude Oil Prices with Macroeconomic & Financial Indices

Raihan Arvi / Fabrizzio Bernie / Pei Yi Chew / Syifa Izzah / Kezia Wijaya

STAT 443 - The University of British Columbia
April 10, 2026



Abstract

Crude oil prices are highly volatile and influenced by a wide range of economic and financial factors, making accurate forecasting a challenging task. This report investigates the effectiveness of an ARMAX model in forecasting U.S. crude oil prices using monthly data. In addition to historical oil prices, several macroeconomic and financial indicators: including the U.S. Dollar Index, OPEC Oil Production Index, and Industrial Production Index, are analyzed and incorporated as exogenous variables.

To ensure stationarity and stabilize variance, all series are transformed using log-differences. Model identification is guided by ACF and PACF diagnostics, while model selection is based on both Akaike Information Criterion (AIC) and out-of-sample one-step-ahead RMSE. Baseline ARIMA models are first evaluated, showing limited predictive power due to the inherent unpredictability of oil price changes. The inclusion of exogenous variables is then examined through cross-correlation analysis and ARMAX modeling.

The results show that the ARMAX model achieves the best overall performance, attaining the lowest AIC and the lowest out-of-sample RMSE among all models considered. The findings suggest that while oil price movements remain difficult to predict, the inclusion of relevant external factors can lead to measurable gains in forecasting accuracy.

1 Introduction

One of the most important commodities traded worldwide that drives everything from daily life to industrial level is crude oil. Crude oil is famously affected by many factors, ranging from political events to economic indices. Bloomberg crude oil price data is used as one of the main measures of oil price movements. However, oil prices are known for their high volatility and sensitivity to economic, financial, and geopolitical shocks.

Evidence shows that COVID-19 caused unusual volatility in crude oil prices, exceeding the levels observed during the Global Financial Crisis, and had a more severe impact on oil markets than on the Dow Jones Industrial Average (DJIA) (Wang, Y. and Ding, X., 2024). While the DJIA is not used in our final model, we include it here as a reference to illustrate broader market volatility. These findings highlight that oil prices are highly sensitive to external economic and financial conditions, especially during periods of increased uncertainty. Moreover, as one of the largest players in the oil and gas sector, the value of the United States dollar may play a significant role in influencing crude oil prices (Shang, J., & Hamori, S., 2024).

Because of this sensitivity, models that rely solely on past oil prices may be insufficient. Incorporating exogenous variables may improve forecasting accuracy. This study examines the possibility of the given scenario. By evaluating ARMAX forecasting performance, this study contributes to understanding oil price predictability.

2 Research Question

Through this study, we will answer the following research question: How does an ARMAX model perform in forecasting Bloomberg crude oil prices?

3 Variable Description

3.1 Response Variable

The primary variable of interest is the crude oil spot price, sourced from Bloomberg at a monthly frequency. The series spans from January 1970 to March 2026, consisting of 675 observations. Prices are recorded in U.S. dollars per barrel and represent the last traded price for each month

3.2 Exogenous Variables

Three macroeconomic indicators are used as exogenous variables, each chosen for its well-documented relationship with oil prices:

1. U.S. Dollar Index (DXY): monthly data from Bloomberg, April 1970 to April 2026 (673 observations). The DXY measures the value of the U.S. Dollar relative to a basket of major foreign currencies. A stronger dollar usually depresses oil prices, as crude is denominated in USD.

2. U.S. Industrial Production Index (INDPRO): monthly data from FRED (Federal Reserve Economic Data), January 1919 to February 2026 (1286 observations). INDPRO captures aggregate real output of the U.S. manufacturing, mining, and utilities sectors. It serves as proxy for domestic demand for energy.

3. OPEC Crude Oil Production (OPCRTOTL): monthly data from Bloomberg, December 1970 to March 2026 (464 observations). Total OPEC output, measured in thousands of barrels per day, reflects supply-side dynamics in the global oil market.

All data sourced from Bloomberg was taken from Bloomberg Terminal accessed at The Canaccord Learning Commons (CLC) in The Sauder School of Bussiness.
We decided to use the data from January 1975 to December 2010 for the training set and the data from January 2011 to January 2026 for the holdout set.

4 Methodology

To construct the ARMAX model, we first transformed the crude oil series and explanatory variables using log-differences so that we obtain more stable and relatively stationary series. We then created the lagged versions of the explanatory variables, so that the forecast equation would not use any unavailable information in the forecast origin. We chose exogenous variables and fit the ARMAX model. We then compare if the ARMAX model is an improvement of the ARMA model based on the rolling 1-step RMSE and AIC.

5 Time Series Diagnostics

5.1 Seasonality(ACF & PACF)

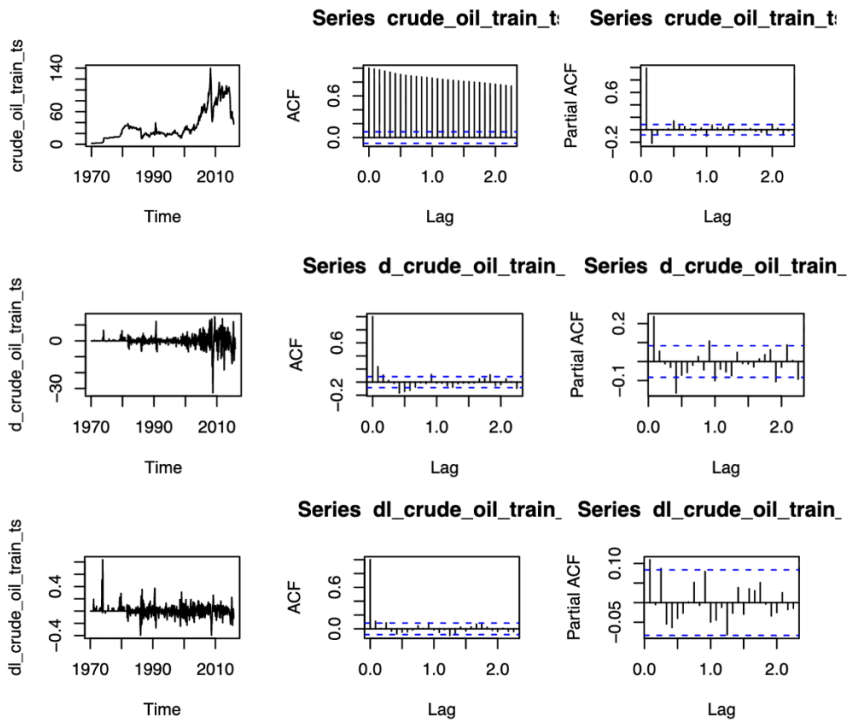


Figure 1: ACF & PACF

An examination of the Autocorrelation Function (ACF) and Partial Autocorrelation Function (PACF) for the Bloomberg crude oil log-return series reveals no statistically significant spikes at the key seasonal intervals (lags 12, 24, and 36). Throughout the training period, the coefficients at these seasonal lags remain consistently within the 95% confidence intervals. This means there is no detectable monthly seasonality in the Bloomberg crude oil log returns over the training periods.

For the first-differenced series (`d_crude_oil_train_ts`), the PACF displays a single significant spike at lag 1 followed by a sharp cutoff, while the ACF tails off gradually. This pattern is indicative of an AR(1) process, suggesting an ARMA(1,0) specification on the price series.

For the log-differenced series (`dl_crude_oil_train_ts`), the ACF exhibits a single significant spike at lag 1 followed by a sharp cutoff, while the PACF tails off gradually. This pattern is characteristic of a MA(1) process, suggesting an ARMA(0,1) specification on the log price series.

5.2 Stationarity (Variogram)

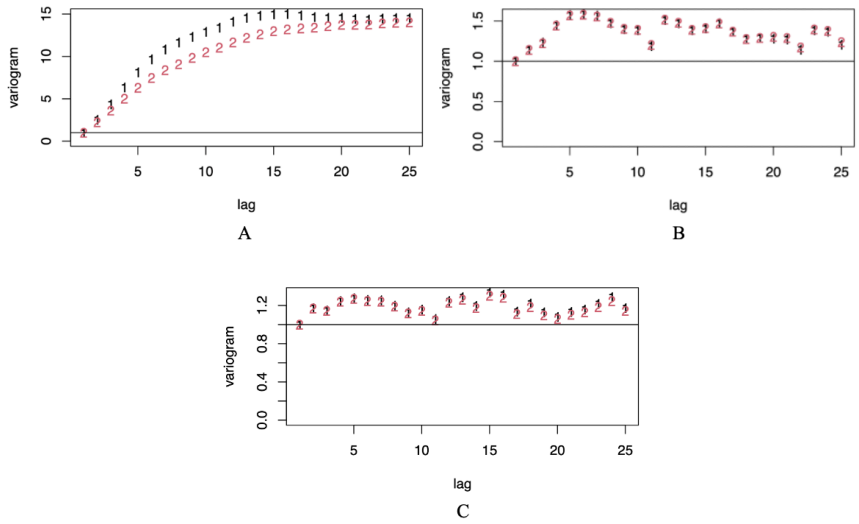


Figure 2: Variogram

For the original level series (`crude_oil_train_ts`), both $\hat{G}_k$ and $\hat{H}_k$ row continuously across all 25 lags. The two estimators does not align, confirming that the original level series is non-stationary.

For the first-differenced series (`d_crude_oil_train_ts`), both $\hat{G}_k$ and $\hat{H}_k$ remain in close agreement across all 25 lags, hovering between 1.1 and 1.5 with no systematic upward trend, confirming that first-differencing is sufficient to induce stationarity.

For the log-differenced series (`dl_crude_oil_train_ts`), $\hat{G}_k$and $\hat{H}_k$ are essentially identical, remaining tightly bounded between 1.0 and 1.2 across all lags, confirming stationarity and representing the most well-behaved transformation of the three.

Since the crude oil time series (y_t) becomes approximately stationary after differencing, $\hat{G}_k$ provides a reliable estimate of the variogram. In contrast, $\hat{H}_k$, which relies on the stationarity assumption, may not accurately estimate the variogram when this assumption is violated. As a result, for non-stationary series, $\hat{G}_k$ and $\hat{H}_k$ can differ substantially.

The log-difference transformation is preferred over first-differencing as crude oil, like most commodity prices, is multiplicative in nature, whereby price fluctuations scale with the price level itself. When prices are at elevated levels, absolute price changes tend to be larger, resulting in heteroscedastic behavior in the first-differenced series. Taking the log-difference compresses this variance, converting proportional changes into additive, and yielding a homoscedastic series that satisfies the constant variance assumption required for ARMA modeling. This is supported by the variogram analysis, where `dl_crude_oil_train_ts` produces a flatter profile proximal to unity compared to the upward-trending variogram of `d_crude_oil_train_ts`, confirming superior stationarity properties.

For the rest of the analysis, we will be focusing only on the log-difference and ditching the difference without log.

6 ARMA Models

We will compare between the ARMA model suggested by `auto.arima` and the ARMA model suggested by looking at the ACF and the PACF plots.

6.1 Using `auto.arima`

We first start with `auto.arima`.

6.1.1 Forecast vs Actual: Price Difference `ARIMA(1,0,0)` on `d_crude_oil_train_ts`

The forecast line remains nearly flat near zero across the holdout period, failing to track the actual price differences which exhibit substantial volatility, including large spikes around time 50 and 120. This indicates that the AR(1) model captures very little of the short-run price dynamics, producing forecasts that are effectively close to the mean. The model is underperforming, particularly during periods of large price shocks. This weak forecasting performance is also reflected in the holdout RMSE of approximately 0.6551. Refer to Figure 17 in Appendix 14.4.1.

6.1.2 Forecast vs Actual - Price Log Difference `ARIMA(0,0,1)` on `dl_crude_oil_train_ts`

The forecast line remains largely flat and close to zero across the holdout period, failing to track the actual price differences which exhibit substantial volatility, including large spikes around time 50 and 120. While the log-differenced forecasts are more stable in scale relative to the price difference forecasts, the MA(1) model still fails to anticipate directional movements of shocks. This limited forecasting ability is also reflected in the holdout RMSE of around 0.1135. Refer to Figure 18 in Appendix 14.4.1.

Both models confirm that crude oil price changes are largely unpredictable from their own past values alone, motivating the subsequent introduction of exogenous variables.

6.2 ARMA Based on PACF/ACF

In the section 5.1, we have identified the ARMA(p,q) model based on the ACF and the PACF plots of the log-differenced series (`dl_crude_oil_train_ts`). The ACF exhibited a sharp cutoff after lag 1 while the PACF tailed off gradually, characteristic of an MA(1) process and suggesting an `ARMA(0,1)` specification. This was subsequently confirmed by `auto.arima`, which selected an `ARIMA(0,0,1)` model with zero mean, yielding an MA(1) coefficient of 0.1459 (`s.e. 0.0486`), an estimated innovation variance of $\hat{\sigma}^2 = 0.0077$, and information criteria of $AIC = -869.70$ and $BIC = -861.57$. The model was evaluated using a rolling one-step-ahead forecasting scheme over the holdout period, yielding an RMSE of 0.11361.

7 Exogenous Variable Analysis

We had chosen several exogenous variables that are potentially correlated with the main time series. In order to see if they are correlated, we will plot their respective cross-correlation.

7.1 Diff. Log U.S. Dollar Index `dl_usd_idx_train_ts`

The log-differenced U.S. Dollar Index fluctuates around zero with relatively stable variance across the entire training period, exhibiting homoscedastic behavior. The series appears stationary and is suitable for use as an exogenous regressor in subsequent modeling.

7.2 Diff. Log Industrial Production Index `dl_ind_train_ts`

The log-differenced Industrial Production Index fluctuates around a slightly positive mean, consistent with long-run productivity growth expressed as a stable monthly growth rate. Variance is largely constant across the sample, though a pronounced negative spike is observed around 2008-2009, attributable to the Global Financial Crisis. Outside of this episode, the series behaves in a stationary and homoscedastic manner, making it appropriate for inclusion as an exogenous variable.

7.3 Diff. Log OPEC Oil Production Index `dl_opec_idx_train_ts`

The log-differenced OPEC oil production series remains tightly bounded and near zero for the majority of the training period, indicating stable and stationary behavior. However, a cluster of large, sharp spikes is evident around 2008-2010, reflecting abrupt OPEC production decisions during the Global Financial Crisis. Outside of this period of elevated volatility, the series is otherwise well-behaved, though the presence of these outliers warrants caution when interpreting cross-correlation results with crude oil prices.

8 Cross-correlation diagnostics for leading variable

We will check the cross correlation between each the extrogenous variables and the target variable.

8.1 CCF: dlog(INDPRO) vs dlog(Bloomberg)

The CCF displays cross-correlations that are largely contained within the confidence bounds across all leads and lags, with no systematic pattern of significant spikes. The lag-1 scatter plot confirms no clear linear relationship between the two variables, suggesting that industrial production growth provides limited predictive power for short-run crude oil price movements.

8.2 CCF: dlog(OPEC) vs dlog(Bloomberg)

Similarly, the CCF shows cross-correlations that are largely within the confidence bounds across all leads and lags, with no systematic pattern of significant spikes. The lag-1 scatter plot displays considerable dispersion with no clear linear trend. Despite the economic intuition that OPEC supply decisions precede price adjustments, the cross-correlation structure does not provide strong statistical evidence of a leading relationship in this training sample.

8.3 CCF: dlog(DXY) vs dlog(Bloomberg)

The CCF exhibits a negative spike at lag 0, reflecting a contemporaneous inverse relationship between U.S. dollar strength and crude oil returns. The lag-1 scatter plot displays a diffuse cloud with no clear linear relationship, indicating that the lagged DXY provides limited predictive power beyond the contemporaneous effect. Nevertheless, the DXY index remains a strong candidate for inclusion as an exogenous variable in an ARIMAX specification given the strength of its contemporaneous cross-correlation.

We used all three variables for the ARMAX model ultimately.

9 ARMAX

To evaluate whether explanatory variables improve short-run forecasting accuracy, we fitted an ARMAX model to the log-differenced crude oil series using lagged industrial production (`ind1`), OPEC production (`opec1`), and the U.S. Dollar Index (`usd1`) as exogenous regressors. Based on the ACF and PACF diagnostics, a low-order MA structure was appropriate, so we considered an ARMAX$(0, 0, 1)$ model. The final fitted model was a regression with ARIMA$(0, 0, 1)$ errors. The estimated MA coefficient was 0.1301, while the

coefficients for industrial production, OPEC production, and the U.S. Dollar Index were 0.3711, -0.0250, and -0.4817, respectively. The negative coefficient for the U.S. Dollar Index is economically reasonable, since oil is priced in U.S. dollars and a stronger dollar is often associated with downward pressure on oil prices.

Model comparison was based on both information criteria and rolling 1-step-ahead holdout forecasts, since the project emphasizes out-of-sample RMSE rather than model selection by AIC alone. The ARMAX model achieved an AIC of -872.8 and an AICc of -872.66, slightly improving on the ARIMA benchmark. In the holdout period, the ARMAX model produced an RMSE of 0.1117, compared with 0.1132 for the ARIMA$(0, 0, 1)$ model. Thus, including lagged explanatory variables led to a modest improvement in out-of-sample forecasting performance.

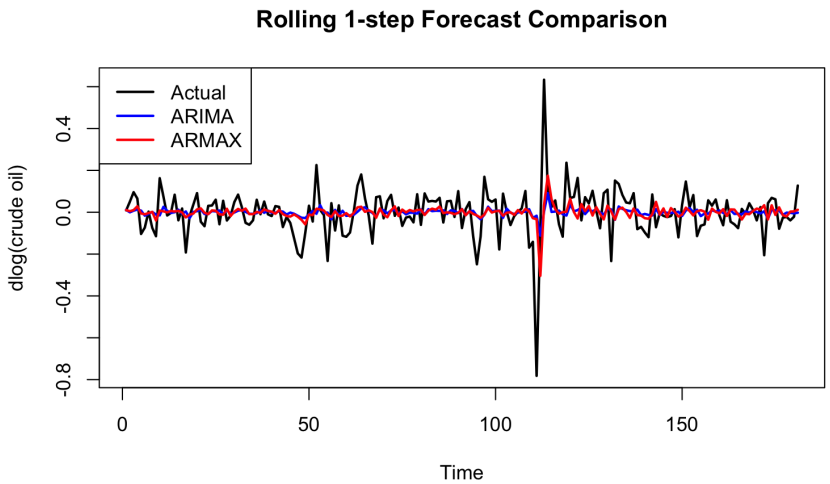


Figure 3: Rolling 1-step forecasts: ARIMA vs ARMAX

Figure 3 shows that the ARMAX forecasts remain close to zero for most of the holdout period, similar to the ARIMA forecasts. This is expected because the response variable is the monthly log return of crude oil prices, which fluctuates around zero. Although the ARMAX model slightly improves forecast accuracy, it still fails to capture large abrupt shocks. This suggests that crude oil returns remain difficult to predict even after adding macroeconomic variables, and that these regressors provide only limited additional leading information. It is possible that the past can not help to forecast the future, if the past is not representative for the future. Forecasting well depends on the assumption that the past is representative of the future (Feedback on Midterm Exam, 2026, p. 2).

Overall, the ARMAX model was preferred to the ARIMA model because it had both a lower AICc and a lower holdout RMSE. However, the improvement was small, so the results should be interpreted as showing only a modest forecasting gain.

10 Simple Forecasting Rules

To benchmark the ARIMA and ARMAX models, we apply four simple forecasting rules to the same holdout (log difference series) set using rolling one-step-ahead predictions.

10.1 Persistence

The persistence rule sets $\hat{y}_{t+1} = y_t$, repeating the most recently observed log return. It requires no estimation but implicitly assumes strong positive autocorrelation. Because the ACF of the log-return series is near zero

at all lags, this assumption is violated. Persistence produces the highest RMSE out of all simple methods at 0.1507. The forecast exhibits a characteristic one-period phase lag, echoing every spike one month late and inflating errors around volatile episodes such as the 2020 oil-price collapse.

10.2 Training-Set Average

This rule issues a constant forecast equal to the unconditional sample mean, $\hat{y}_{t+1} = \bar{y}_{\text{train}} \approx 0.0049$. Despite its inability to capture structure, it manages to achieve an RMSE of 0.1151, which is lower than persistence. This reflects the near-white-noise behaviour of log returns. By forecasting close to the long-run mean avoids the large phase-lag penalties that persistence incurs.

10.3 Simple Exponential Smoothing

Simple exponential smoothing updates a level estimate each period via $L_t = \alpha\, y_t + (1-\alpha)L_{t-1}$, with $\alpha = 0.5$ and $L_0 = \bar{y}_{\text{train}}$. The forecast is $\hat{y}_{t+1} = L_t$. By placing equal weight on the latest observation and the accumulated history, SES lies in between the full reactivity of persistence and the stillness of the training-set average. Its holdout RMSE of 0.1323 confirms this: partial adaptation helps relative to persistence, but the added responsiveness does not help when compared with the flat mean forecast.

10.4 Linear in Previous Observation

This rule fits $\hat{y}_{t+1} = \hat{\beta}_0 + \hat{\beta}_1\, y_t$ by OLS on the training set, yielding $\hat{\beta}_0 = 0.0042$ and $\hat{\beta}_1 = 0.1376$. The near-zero intercept and small positive slope indicate it has weak linear dependence on the lagged return. The holdout RMSE of 0.1140 is only slightly better than the training-set average, it is effective identical. This confirms that the lagged return contributes negligible predictive power beyond the unconditional mean

10.5 Comparison with ARIMA and ARMAX

Compared with the simple forecasting rules, the ARIMA and ARMAX models perform slightly better in general. The most competitive simple methods were the training-set average and linear regression rules, with RMSE values of approximately 0.1151 and 0.1140, respectively. These were both slightly worse than the ARIMA model, which has an RMSE of around 0.1132, and the ARMAX model, which has the lowest RMSE of around 0.1117.

This suggests that monthly crude oil log returns are hard to predict, since even simple benchmark methods perform reasonably well. However, the ARIMA and ARMAX models still pick up a small amount of additional useful information. However, the ARIMA and ARMAX models still extract a small amount of additional predictive information. Among all methods considered, ARMAX performs best, indicating that lagged macroeconomic variables provide a modest improvement in short-run forecasting accuracy.

11 Conclusion

The results show that the ARMAX model outperforms all other models, achieving the lowest rolling RMSE (0.1117) and the lowest AIC (-872.8009). This indicates that ARMAX provides both the most accurate out-of-sample forecasts and the best in-sample model fit.

In comparison, ARMA models using log-differenced data perform similarly but slightly worse than ARMAX, suggesting that incorporating exogenous variables contributes additional predictive power. However, the improvement is slight, indicating that most of the predictive structure is already captured by the autoregressive and moving average components.

Benchmark models such as Persistence and Exponential Smoothing perform noticeably worse, confirming that simple methods are insufficient for capturing the dynamics of the data. Linear Regression performs competitively however does not surpass ARMAX.

The summary table of all the fitted models could be found in the 14.1 Comparison of Fitted Models.

Overall, the findings suggest that the ARMAX model is the best choice based on AIC and rolling 1-step RMSE in this problem.

12 Reference

Federal Reserve Bank of St. Louis. (2026). Dow Jones Industrial Average (DJIA). FRED, Federal Reserve Bank of St. Louis. https://fred.stlouisfed.org/series/DJIA

Federal Reserve Bank of St. Louis. (2026). Trade Weighted U.S. Dollar Index: Broad, Goods and Services (DTWEXBGS). FRED, Federal Reserve Bank of St. Louis. https://fred.stlouisfed.org/series/DTWEXBGS

Shang, J., & Hamori, S. (2024). The response of oil-importing and oil-exporting countries' macroeconomic aggregates to crude oil price shocks: Some international evidence. Eurasian Economic Review, 14(4), 933-980. doi:https://doi.org/10.1007/s40822-024-00281-z

STAT 443. (2026). Feedback on midterm exam (March 5). University of British Columbia.

Wang, Y. and Ding, X. (2024) Dependence Structure of the US Dollar Index and Crude Oil Prices: A Regime-Switching Copula Approach. Journal of Mathematical Finance, 14, 172-189. doi: 10.4236/jmf.2024.142009.

13 Contributions

13.1 The Team

The team was formed from a friendship of 5 people from Southeast Asia, particularly nearby the Melacca Peninsula and the Sunda Strait. We became best friends during this (STAT 443) class, since we have a similar taste in humor, sport (badminton), and food (we like Bigway) and plan to travel around the world tomorrow together.

13.2 Name Ordering

Name ordering is based on the alphabetical order of the last names.

13.3 Member Contributions

1. Raihan Arvi
   - Coming out with the idea of the project and economic interpretation
   - Coding ARMA, plots, and model diagnostics
   - Finding data in Bloomberg Terminal
   - Double-checking the project
2. Fabrizzio Bernie
   - Coding simple rules
   - Report writing
3. Pei Yi Chew
   - Making CCL plots
   - interpreting plots
   - Report writing
4. Syifa Nadhira Nurul Izzah
   - Help finding journals for proposal and datasets for the project
   - Coding ARMAX function
   - Writing ARMAX for the report
   - Double-checking the project
5. Kezia Wijaya
   - Report writing
   - Coding data loading and forecasting
   - Coming out with the idea to obtain macroeconomic indicators data through Bloomberg and finding data in Bloomberg Terminal

14 Appendix

14.1 Comparison of Fitted Models

Model	**Rolling RMSE**	**AIC**
ARMA (diff)	6.5500040	2298.7546
ARMA (log diff - auto)	0.1134567	-869.7040
ARMA (log diff - manual)	0.1136074	-868.7228
ARIMA (baseline)	0.1131622	-869.7040
ARMAX	0.1117057	-872.8009
Persistence	0.1506962	NA
Average	0.1151330	NA
Exponential Smoothing	0.1323217	NA
Linear Regression	0.1139725	NA

Table 1: Model Comparison Based on Rolling RMSE and AIC

14.2 Figures

14.2.1 Bloomberg Spot Oil Level Price

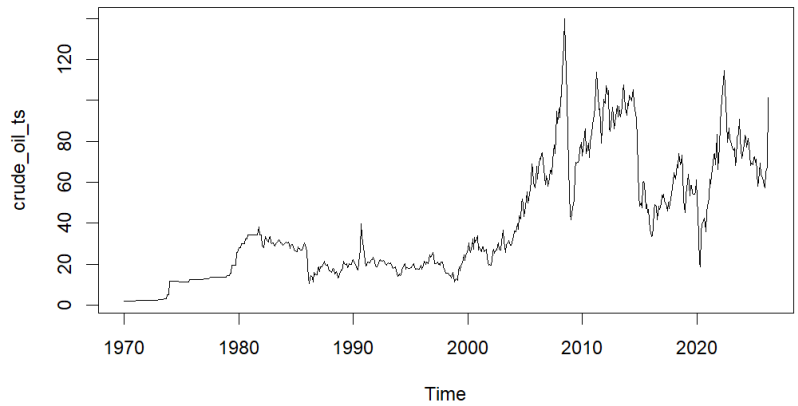


Figure 4: Bloomberg spot crude oil price, 1970–2026

14.2.2 Bloomberg Spot Oil Diagnostic Plots

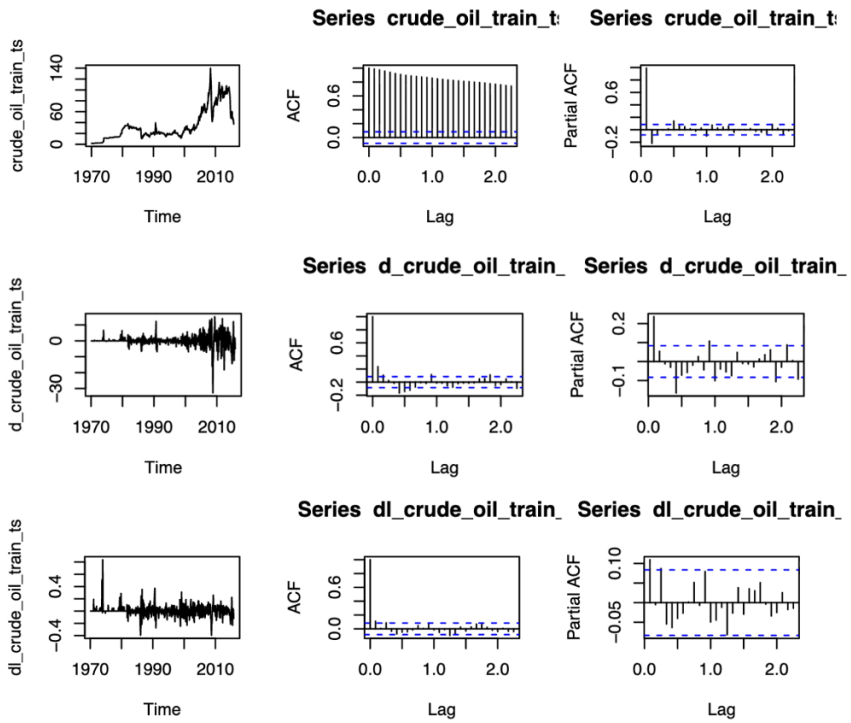


Figure 5: ACF & PACF

14.2.3 Bloomberg Spot Oil Variograms

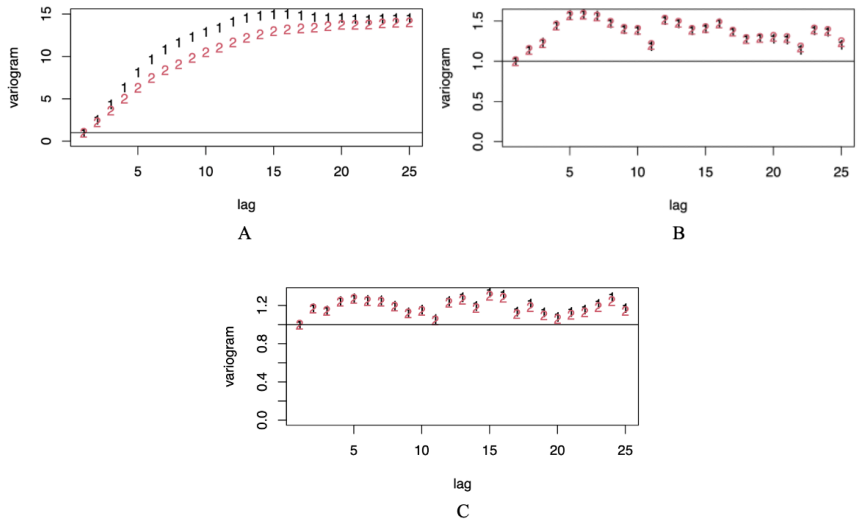


Figure 6: Variogram

14.3 Extrogenous Variables Price Level PLots

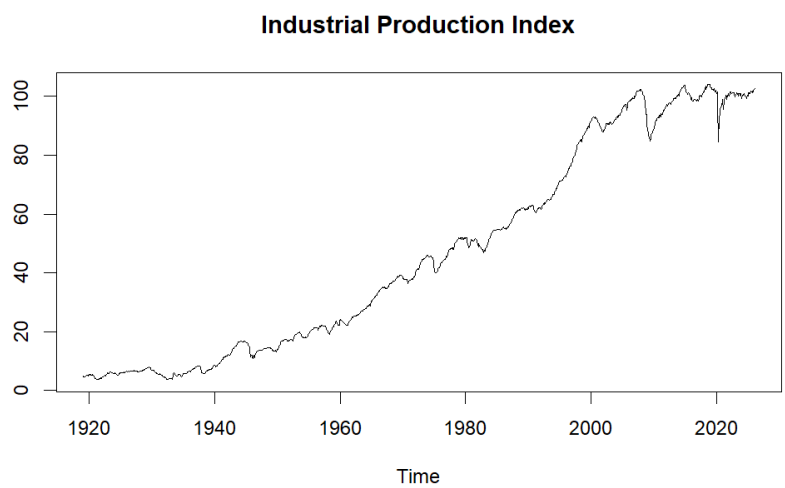


Figure 7: Industrial Production Index

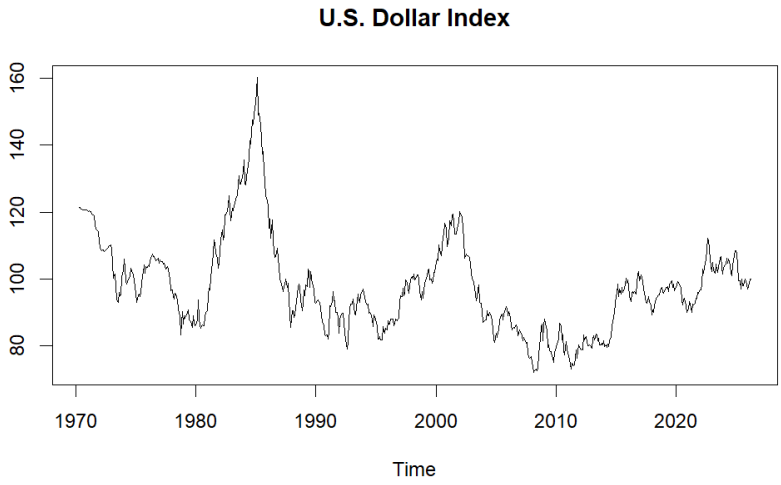


Figure 8: USD Price Index

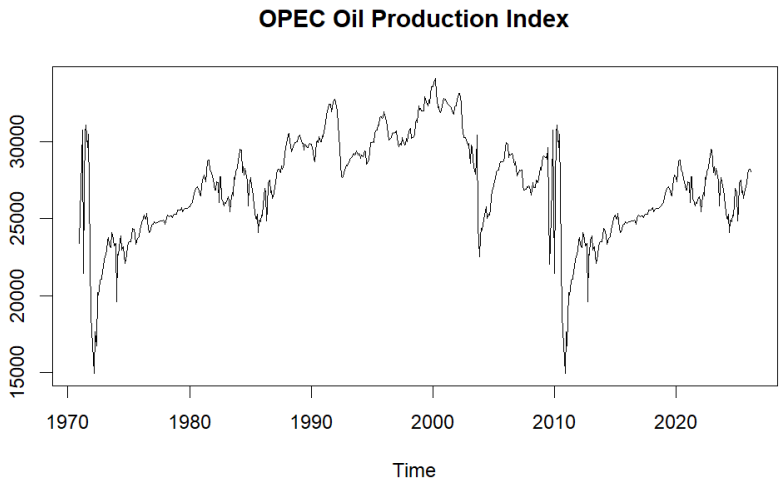


Figure 9: OPEC Oil Production Index

14.3.1 Log-differencing Plots

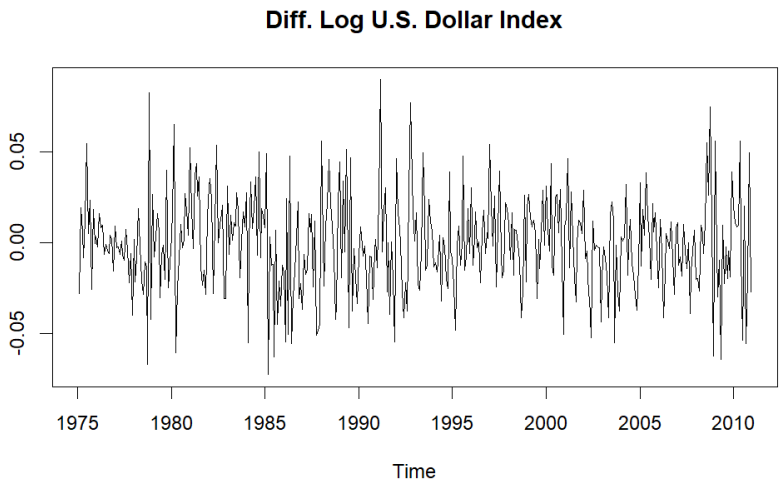


Figure 10: Log-Differencing U.S. Dollar Index

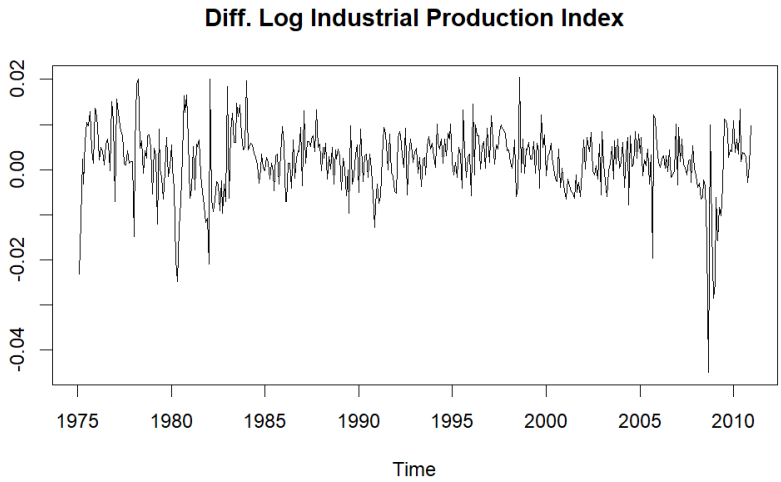


Figure 11: Log-Differencing Industrial Production Index

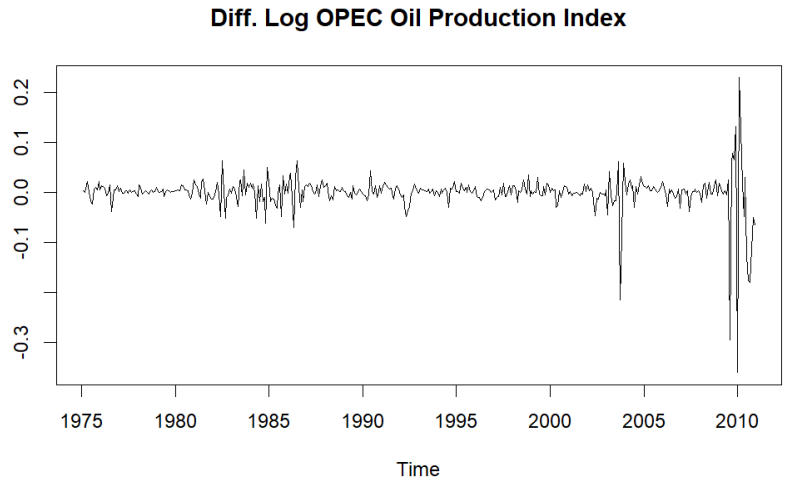


Figure 12: Log-Differencing Oil Production Index

14.4 CCF Plots

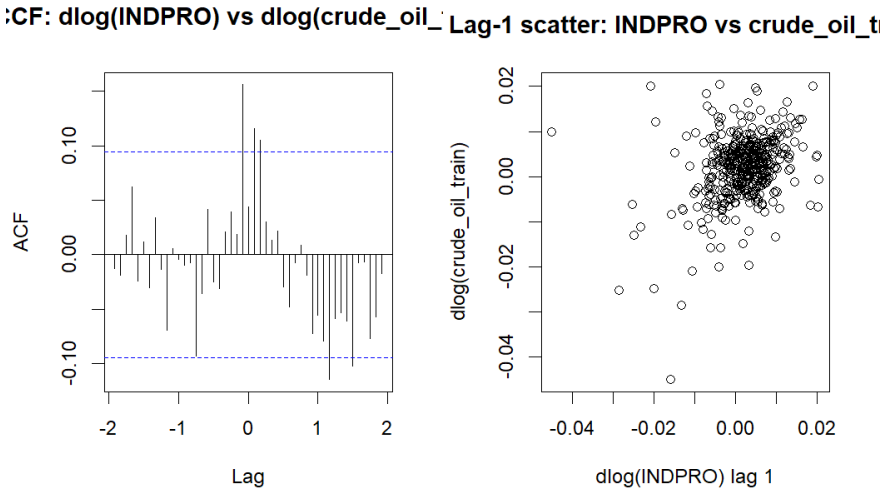


Figure 13: CCF Oil vs Industrial Production Index

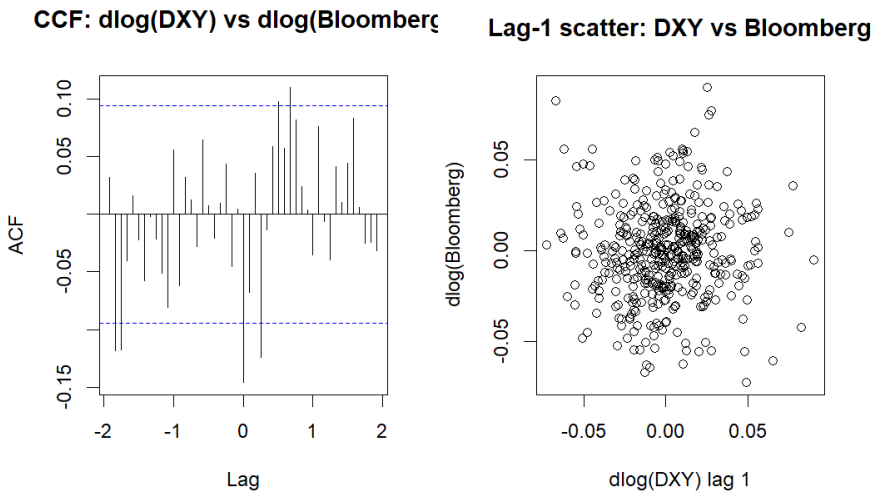


Figure 14: CCF Oil vs USD Price Index

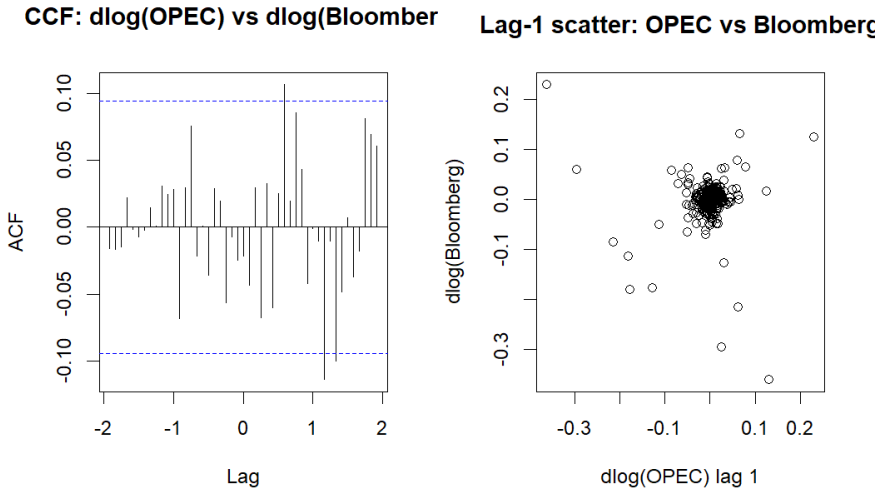


Figure 15: CCF Oil vs OPEC Oil Production Index

14.4.1 Rolling 1-Step Forecasts

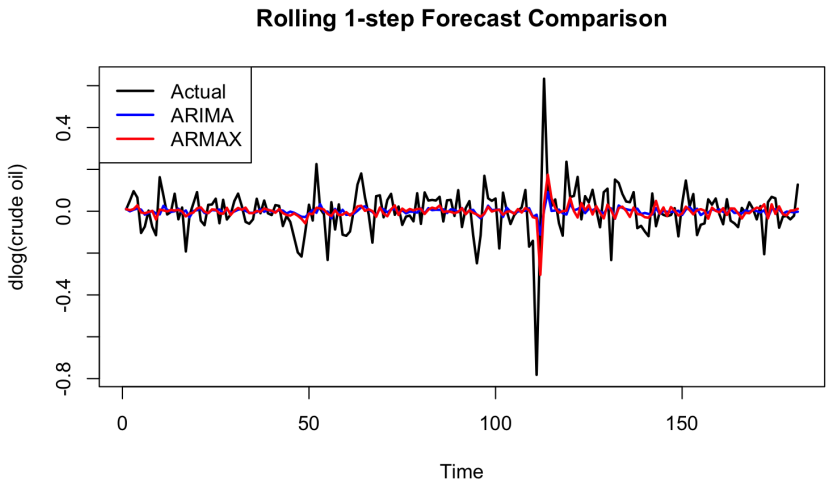


Figure 16: Rolling 1-step forecasts: ARIMA vs ARMAX

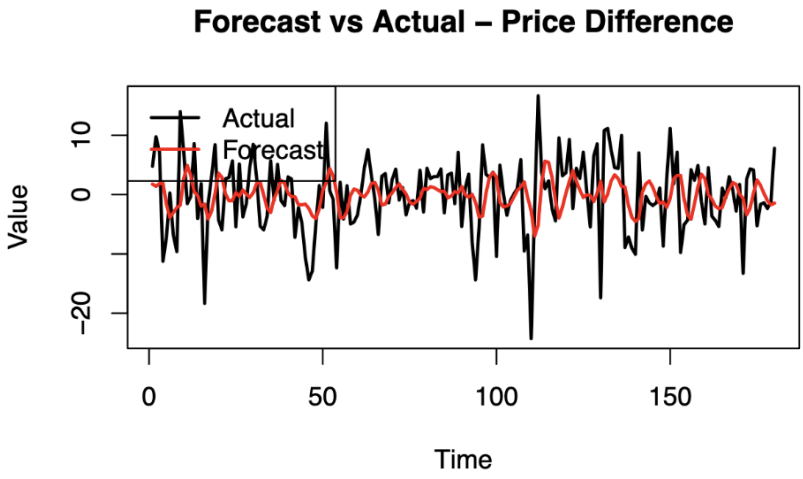


Figure 17: Forecast VS Actual - Price Difference

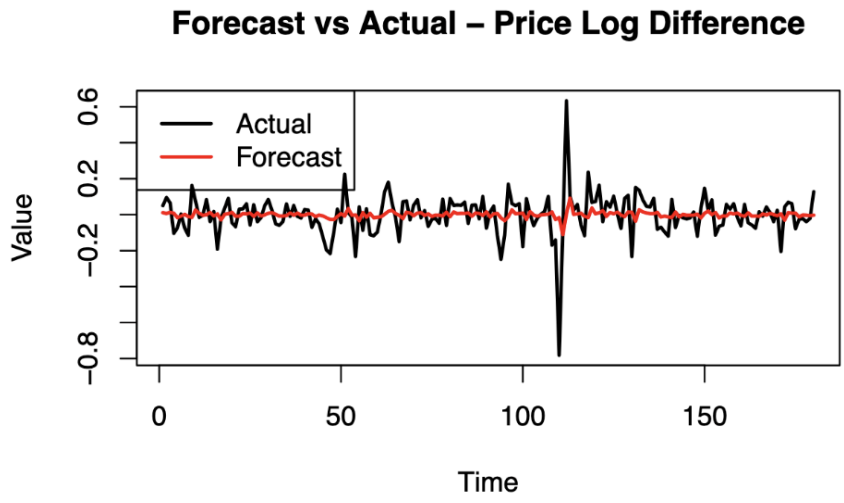


Figure 18: Forecast VS Actual - Price Log Difference

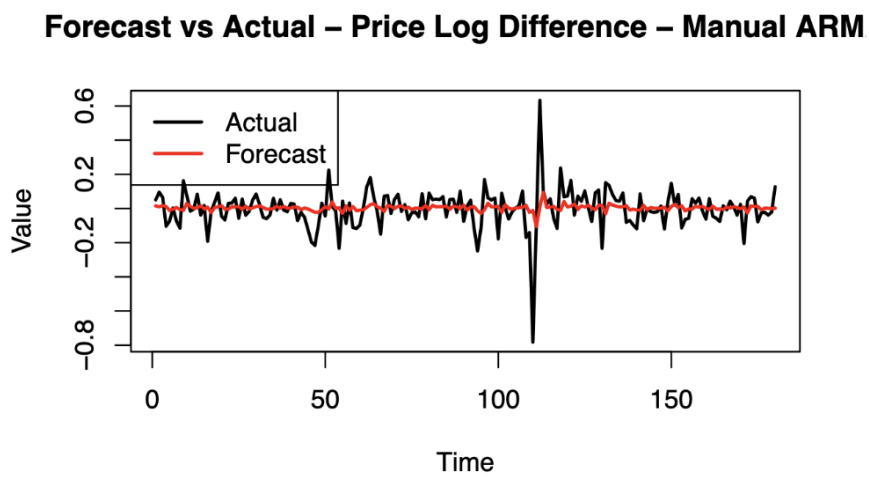


Figure 19: Forecast VS Actual - Price Log Difference - Manual ARM

14.4.2 Simple Rules Rolling 1-Step Forecasts on Holdout Set

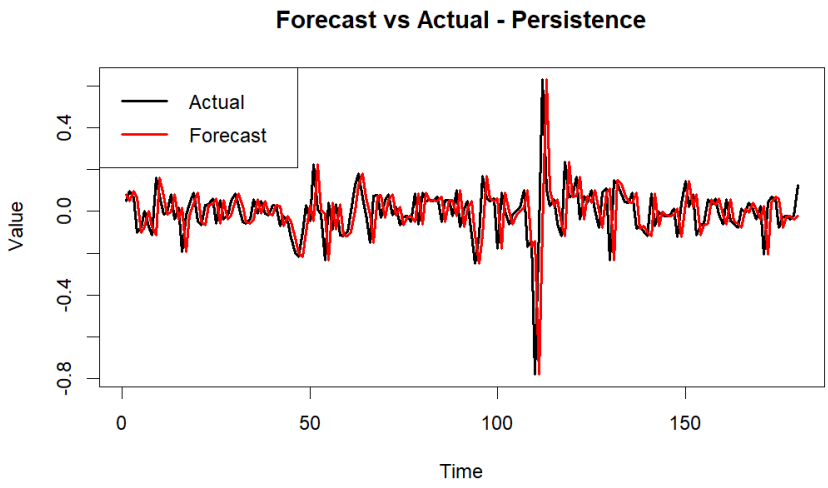


Figure 20: Persistence

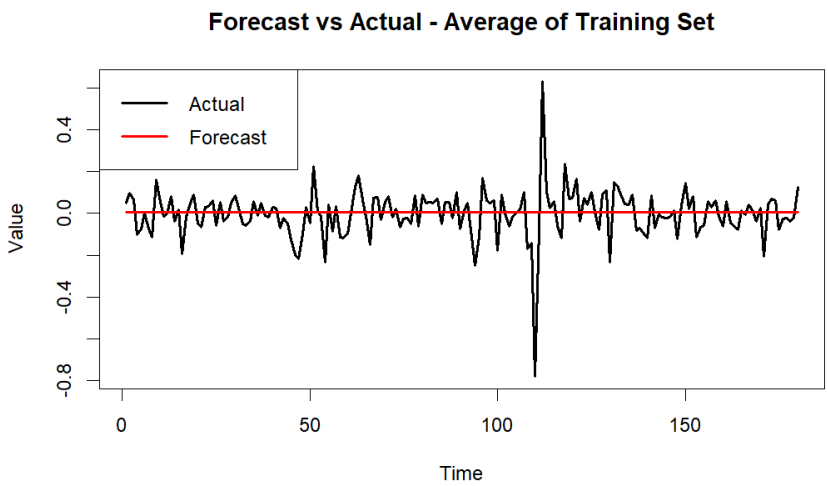


Figure 21: Average Mean Forecast

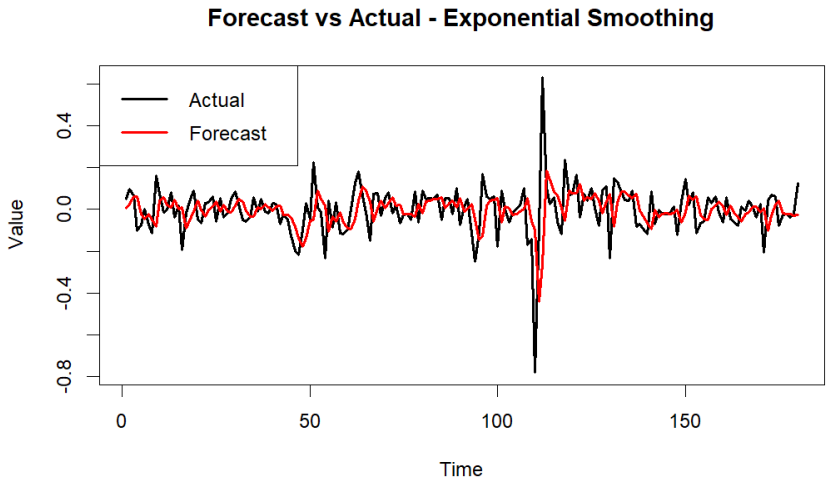


Figure 22: Exponential Smoothing

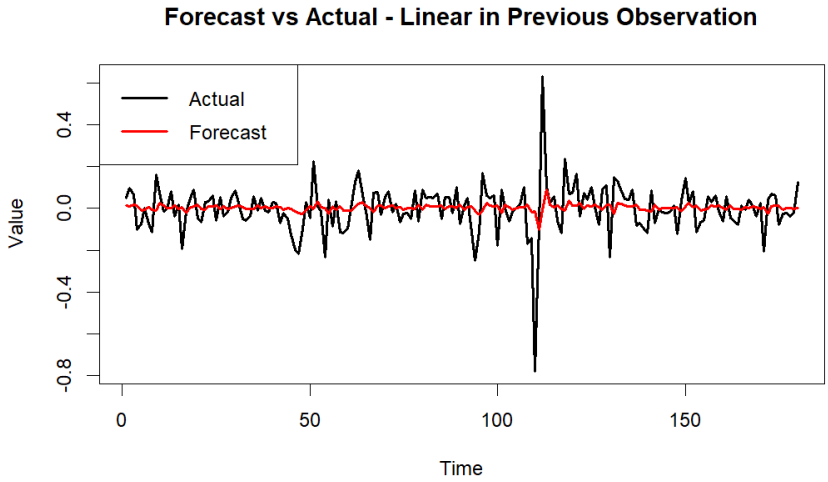


Figure 23: Linear Regression Forecast

14.5 R Code

```
---
title: "STAT-443-Project"
format: pdf
editor: visual
---

```{r}
library(forecast)
```

Functions

Some of the functions below were taken from labs or lectures.

```{r}
# Function for variogram as defined in
# Bisgaard and Kulahci (2011). Time Series Analysis and Forecasting by Example, Wiley
variogram = function(y, lagmax=10, iprint=F)
{ G = rep(1,lagmax)
n = length(y)
if(lagmax>n) { lagmax = n-2 }
y1 = y[-1]; y2 = y[-n]
d1 = y1-y2; denom = var(d1)
for(k in 2:lagmax)
{ y1 = y[(k+1):n]; y2 = y[1:(n-k)]
dk = y1-y2
numer = var(dk)
G[k] = numer/denom
}
# H is the variogram assuming a stationary time series
H = rep(1,lagmax)
ac = c(acf(y,plot=F,lag.max=lagmax)$acf)
H = (1-ac[-1])/(1-ac[2])
if(iprint)
{ print(cbind(G,H))
mx = max(cbind(G,H))
matplot(1:lagmax,cbind(G,H),ylim=c(0,mx),ylab="variogram", xlab="lag")
abline(h=1)
}
list(G=G, H=H)
}
```

```{r}
persist_fc = function(train, holdout, iprint = FALSE) {
  ntrain = length(train)
  nh = length(holdout)
  sse = 0
  fc_vec = numeric(nh)
  fc = train[ntrain]
  yt = holdout[1]
  fc_vec[1] = fc
```
```

```
  fcerror = yt - fc
  sse = sse + (fcerror^2)
  for (i in 2:nh) {
    yt = holdout[i]
    fc = holdout[i-1]
    fc_vec[i] = fc
    fcerror = yt - fc
    sse = sse + (fcerror^2)
  }
  return(list(rmse = sqrt(sse/nh), fc = fc_vec))
}

# Linear in the previous observation
# b0,b1 from least squares or another estimation method
linreg_fc = function(train_last, holdout, b0,b1)
{ nh = length(holdout)
  previous = c(train_last, holdout[-nh])
  fc_vec = b0+b1*previous
  rmse = sqrt(mean((holdout-fc_vec)^2))
  return(list(rmse=rmse, fc=fc_vec))
}

# Average Forecast
avg_fc = function(train, holdout) {
  nh = length(holdout)
  avg = mean(train)
  sse = 0
  fc_vec = numeric(nh)
  for (i in 1:nh) {
    fc = avg
    fc_vec[i] = fc
    yt = holdout[i]
    fcerror = yt - fc
    sse = sse + (fcerror^2)
  }
  return(list(rmse = sqrt(sse/nh), fc = fc_vec))
}

# Simple Exponential Smoothing
esm_fc = function(train, holdout, alpha, level, iprint = FALSE) {
  nh = length(holdout)
  sse = 0
  fc_vec = numeric(nh)
  L = level
  for (i in 1:nh) {
    fc = L
    fc_vec[i] = fc
    yt = holdout[i]
    fcerror = yt - fc
    sse = sse + (fcerror^2)
    L = alpha * yt + (1 - alpha) * L
  }
  return(list(rmse = sqrt(sse / nh), fc = fc_vec))
}
```

```
```

```{r}
plot_forecast = function(holdout, fc, model_name = "Model") {

  x = seq_along(holdout)

  plot(x, holdout, type = "l", col = "black", lwd = 2,
       main = paste("Forecast vs Actual —", model_name),
       ylab = "Value", xlab = "Time")

  lines(x, fc, col = "red", lwd = 2)

  legend("topleft",
         legend = c("Actual", "Forecast"),
         col = c("black", "red"),
         lty = 1, lwd = 2)
}
```

```{r}
arima_rolling_fc = function(fitted_model, holdout, iprint = FALSE) {
  nh = length(holdout)
  sse = 0
  fc_vec = numeric(nh)

  # Extract the training data from the fitted model
  train_data = fitted_model$x
  current_data = train_data

  for (i in 1:nh) {
    # Forecast one step ahead using current model
    fc = forecast(Arima(current_data, model = fitted_model), h = 1)$mean[1]
    fc_vec[i] = fc

    yt = holdout[i]
    fcerror = yt - fc
    sse = sse + (fcerror^2)

    # Expand training data with actual observed value
    current_data = c(current_data, yt)
  }

  return(list(rmse = sqrt(sse / nh), fc = fc_vec))
}
```

```{r}
rolling_armax_fixed = function(y_train, x_train, y_holdout, x_holdout,
                               order, include_mean = TRUE) {
  nh = length(y_holdout)
  fc_vec = numeric(nh)

  y_hist = y_train
```

```r
  x_hist = as.matrix(x_train)
  x_holdout = as.matrix(x_holdout)

  for (i in 1:nh) {
    fit = forecast::Arima(
      y_hist,
      order = order,
      xreg = x_hist,
      include.mean = include_mean,
      method = "ML"
    )

    newx = matrix(x_holdout[i, ], nrow = 1)

    fc_vec[i] = forecast::forecast(
      fit,
      xreg = newx,
      h = 1
    )$mean[1]

    y_hist = c(y_hist, y_holdout[i])
    x_hist = rbind(x_hist, newx)
  }

  rmse = sqrt(mean((y_holdout - fc_vec)^2))
  list(rmse = rmse, fc = fc_vec)
}
```

Load Data

```{r}
# Load Spot Crude Oil Price
crude_oil = read.csv("data/bloomberg_oil_spot_monthly.tsv", header = TRUE, sep = "\t")
crude_oil$Date = as.Date(crude_oil$Date)
crude_oil = crude_oil[order(crude_oil$Date), ]

crude_oil_ts = ts(
  crude_oil$Last.Price,
  start = c(as.numeric(format(min(crude_oil$Date), "%Y")),
            as.numeric(format(min(crude_oil$Date), "%m"))),
  frequency = 12
)

head(crude_oil_ts)
```

Split Training - Holdout

```{r}
crude_oil_train_ts = window(crude_oil_ts, start = c(1975,1), end = c(2010,12))
crude_oil_holdout_ts = window(crude_oil_ts, start = c(2011,1), end = c(2026,1))

d_crude_oil_train_ts = diff(crude_oil_train_ts)
```

```
d_crude_oil_holdout_ts = diff(crude_oil_holdout_ts)

dl_crude_oil_ts = diff(log(crude_oil_ts))
dl_crude_oil_train_ts = diff(log(crude_oil_train_ts))
dl_crude_oil_holdout_ts = diff(log(crude_oil_holdout_ts))
```

Diagnostic Plots

```{r}
plot(crude_oil_ts)
```

```{r}
par(mfrow = c(2,3))

plot.ts(crude_oil_train_ts)
acf(crude_oil_train_ts)
pacf(crude_oil_train_ts)

plot.ts(d_crude_oil_train_ts)
acf(d_crude_oil_train_ts)
pacf(d_crude_oil_train_ts)

plot.ts(dl_crude_oil_train_ts)
acf(dl_crude_oil_train_ts)
pacf(dl_crude_oil_train_ts)
```

```{r}
variogram(crude_oil_train_ts, lagmax=25, iprint=T)
variogram(d_crude_oil_train_ts, lagmax=25, iprint=T)
variogram(dl_crude_oil_train_ts, lagmax=25, iprint=T)
```

We first try ARMA

ARMA on difference

```{r}
dwti_arma = auto.arima(d_crude_oil_train_ts, seasonal=F, stationary=T)
print(dwti_arma)
```

Rolling 1-step forward

```{r}
dwti_arma_rolling = arima_rolling_fc(dwti_arma, d_crude_oil_holdout_ts)
dwti_arma_rolling$rmse
```

ARMA on log return

```{r}
```

```
dlwti_arma = auto.arima(dl_crude_oil_train_ts, seasonal=F, stationary=T)
print(dlwti_arma)
```

Rolling 1-step forward

```{r}
dlwti_arma_rolling = arima_rolling_fc(dlwti_arma, dl_crude_oil_holdout_ts)
dlwti_arma_rolling$rmse
```

ARMA based on ACF/PACF

```{r}
dlwti_arma_manual = Arima(dl_crude_oil_train_ts, order = c(0, 0, 1))
print(dlwti_arma_manual)
```

Rolling 1-step forward

```{r}
dlwti_arma_manual_rolling = arima_rolling_fc(dlwti_arma_manual, dl_crude_oil_holdout_ts)
dlwti_arma_manual_rolling$rmse
```

Plots

```{r}
plot_forecast(d_crude_oil_holdout_ts, dwti_arma_rolling$fc, "Price Difference")
plot_forecast(dl_crude_oil_holdout_ts, dlwti_arma_rolling$fc, "Price Log Difference")
plot_forecast(dl_crude_oil_holdout_ts, dlwti_arma_manual_rolling$fc, "Price Log Difference
```

Extrogenous variables

Load data

```{r}
# USD index
usd_idx = read.csv("data/dxy_monthly.tsv", header = TRUE, sep = "\t")
usd_idx$Date = as.Date(usd_idx$Date)
usd_idx = usd_idx[order(usd_idx$Date), ]

usd_idx_ts = ts(
  usd_idx$Last.Price,
  start = c(1970,04),
  end = c(2026,04),
  frequency = 12
)
usd_idx_train_ts = window(usd_idx_ts, start = c(1975,1), end = c(2010,12))
usd_idx_holdout_ts = window(usd_idx_ts, start = c(2011,1), end = c(2026,1))

# Industrial production
ind_prod = read.csv("data/INDPRO.csv")
```

```
ind_prod$observation_date = as.Date(ind_prod$observation_date)
ind_prod_ts = ts(ind_prod$INDPRO, start = c(1919,1), end = c(2026,2), frequency = 12)
ind_prod_train_ts = window(ind_prod_ts, start = c(1975,1), end = c(2010,12))
ind_prod_holdout_ts = window(ind_prod_ts, start = c(2011,1), end = c(2026,1))

# OPEC oil production
opec_prod = read.csv("data/opcrtotl_monthly.tsv", header = TRUE, sep = "\t")
opec_prod$Date = as.Date(opec_prod$Date)
opec_prod = opec_prod[order(opec_prod$Date), ]

opec_prod_ts = ts(
  opec_prod$Last.Price,
  start = c(1970,12),
  end = c(2026,03),
  frequency = 12
)
opec_prod_train_ts = window(opec_prod_ts, start = c(1975,1), end = c(2010,12))
opec_prod_holdout_ts = window(opec_prod_ts, start = c(2011,1), end = c(2026,1))
```

```{r}
plot(usd_idx_ts, main="U.S. Dollar Index", ylab="")
plot(ind_prod_ts, main="Industrial Production Index", ylab="")
plot(opec_prod_ts, main="OPEC Oil Production Index", ylab="")
```

Transform log-differencing

```{r}
d_usd_ts = diff(usd_idx_ts)
dl_usd_idx_ts = diff(log(usd_idx_ts))
dl_usd_idx_train_ts = diff(log(usd_idx_train_ts))

d_ind_ts = diff(ind_prod_ts)
dl_ind_ts = diff(log(ind_prod_ts))
dl_ind_train_ts = diff(log(ind_prod_train_ts))

d_opec_ts = diff(opec_prod_ts)
dl_opec_prod_ts = diff(log(opec_prod_ts))
dl_opec_idx_train_ts = diff(log(opec_prod_train_ts))
```

Plot

```{r}
#plot(dl_usd_idx_train_ts, main="Difference U.S. Dollar Index", ylab="")
plot(dl_usd_idx_train_ts, main="Diff. Log U.S. Dollar Index", ylab="")

#plot(d_ind_train_ts, main="Difference Industrial Production Index", ylab="")
plot(dl_ind_train_ts, main="Diff. Log Industrial Production Index", ylab="")

#plot(d_opec_idx_train_ts, main="Difference Industrial Production Index", ylab="")
plot(dl_opec_idx_train_ts, main="Diff. Log OPEC Oil Production Index", ylab="")
```

Diagnostics: variogram, ACF, PACF

```{r}
par(mfrow=c(1,2))
acf(dl_crude_oil_train_ts,
    main = "ACF: dlog(crude_oil_train)")
pacf(dl_crude_oil_train_ts,
     main = "PACF: dlog(crude_oil_train)")
```

ARMAX

Cross-correlation diagnostics for leading variables

```{r}
par(mfrow=c(1,2))
ccf(dl_ind_train_ts, dl_crude_oil_train_ts, main="CCF: dlog(INDPRO) vs dlog(crude_oil_train
plot(stats::lag(dl_ind_train_ts, -1), dl_ind_train_ts,
     xlab="dlog(INDPRO) lag 1", ylab="dlog(crude_oil_train)",
     main="Lag-1 scatter: INDPRO vs crude_oil_train")
```

```{r}
par(mfrow=c(1,2))
ccf(dl_opec_idx_train_ts, dl_crude_oil_train_ts, main="CCF: dlog(OPEC) vs dlog(Bloomberg)"
plot(stats::lag(dl_opec_idx_train_ts, -1), dl_opec_idx_train_ts,
     xlab="dlog(OPEC) lag 1", ylab="dlog(Bloomberg)",
     main="Lag-1 scatter: OPEC vs Bloomberg")
```

```{r}
par(mfrow=c(1,2))
ccf(dl_usd_idx_train_ts, dl_crude_oil_train_ts, main="CCF: dlog(DXY) vs dlog(Bloomberg)")
plot(stats::lag(dl_usd_idx_train_ts, -1), dl_usd_idx_train_ts,
     xlab="dlog(DXY) lag 1", ylab="dlog(Bloomberg)",
     main="Lag-1 scatter: DXY vs Bloomberg")
```

Align data

```{r}
df_all = ts.intersect(
  oil  = dl_crude_oil_ts,
  ind  = dl_ind_ts,
  opec = dl_opec_prod_ts,
  usd  = dl_usd_idx_ts
)

start(df_all)
end(df_all)
```

Split Train/Holdout

```{r, warning=FALSE}
train_all = window(df_all, start = c(1975, 2), end = c(2010, 12))
holdout_all = window(df_all, start = c(2011, 1), end = c(2026, 1))

y_train = train_all[, "oil"]
y_holdout = holdout_all[, "oil"]

xreg_train = as.matrix(train_all[, c("ind", "opec", "usd")])
xreg_holdout = as.matrix(holdout_all[, c("ind", "opec", "usd")])
```

ARIMA baseline

```{r, warning=FALSE}
# ARIMA baseline
arima_model = auto.arima(
  y_train,
  seasonal = FALSE,
  stationary = TRUE
)

print(arima_model)

arima_res = arima_rolling_fc(arima_model, y_holdout)
```

ARMAX candidate model

```{r, warning=FALSE}
armax_model = auto.arima(
  y_train,
  xreg = xreg_train,
  seasonal = FALSE,
  stationary = TRUE
)

print(armax_model)
```

extract nonseasonal ARMA order

```{r}
order_armax = c(armax_model$arma[1], 0, armax_model$arma[2])
include_mean_armax = any(names(armax_model$coef) %in% c("intercept", "mean"))
```

Rolling 1-step ARMAX

```{r, warning=FALSE}
armax_res = rolling_armax_fixed(
  y_train = y_train,
  x_train = xreg_train,
```

```
  y_holdout = y_holdout,
  x_holdout = xreg_holdout,
  order = order_armax,
  include_mean = include_mean_armax
)
```

RMSE comparison (ARIMA vs ARMAX)

```{r, warning=FALSE}
cat("ARIMA RMSE:", arima_res$rmse, "\n")
cat("ARMAX RMSE:", armax_res$rmse, "\n")
```

Plot forecasts

```{r}
plot(as.numeric(y_holdout), type = "l", col = "black", lwd = 2,
     main = "Rolling 1-step Forecast Comparison",
     ylab = "dlog(crude oil)", xlab = "Time")

lines(as.numeric(arima_res$fc), col = "blue", lwd = 2)
lines(as.numeric(armax_res$fc), col = "red", lwd = 2)

legend("topleft",
       legend = c("Actual", "ARIMA", "ARMAX"),
       col = c("black", "blue", "red"),
       lty = 1, lwd = 2)
```

Simple Forecasting Rules

Persistence

```{r}
persist_result = persist_fc(dl_crude_oil_train_ts, dl_crude_oil_holdout_ts)
persist_result$rmse
```

```{r}
plot_forecast(dl_crude_oil_holdout_ts, persist_result$fc, "Persistence")
```

Average of Training Set

```{r}
avg_result = avg_fc(dl_crude_oil_train_ts, dl_crude_oil_holdout_ts)
avg_result$rmse
```

```{r}
plot_forecast(dl_crude_oil_holdout_ts, avg_result$fc, "Average of Training Set")
```

```
```

Exponential Smoothing

```{r}
level = mean(dl_crude_oil_train_ts)
alpha = 0.5
esm_result = esm_fc(dl_crude_oil_train_ts, dl_crude_oil_holdout_ts, alpha, level)
esm_result$rmse
```
```{r}
plot_forecast(dl_crude_oil_holdout_ts, esm_result$fc, "Exponential Smoothing")
```

Linear in Previous Observation

```{r}
n_train = length(dl_crude_oil_train_ts)
yt = dl_crude_oil_train_ts[2:n_train]
yt1 = dl_crude_oil_train_ts[1:(n_train - 1)]
lm_fit = lm(yt ~ yt1)
print(lm_fit)
```

```{r}
linreg_result = linreg_fc(
  train_last = dl_crude_oil_train_ts[n_train],
  holdout = dl_crude_oil_holdout_ts,
  b0 = coef(lm_fit)[1],
  b1 = coef(lm_fit)[2]
)
linreg_result$rmse
```

```{r}
plot_forecast(dl_crude_oil_holdout_ts, linreg_result$fc,
              "Linear in Previous Observation")
```

Comparison

```{r}
# Rolling RMSE + AIC

model_names = c(
  "ARMA (diff)",
  "ARMA (log diff — auto)",
  "ARMA (log diff — manual)",
  "ARIMA (baseline)",
  "ARMAX",
  "Persistence",
  "Average",
  "Exponential Smoothing",
  "Linear Regression"
```

```
)

rmse_values = c(
  dwti_arma_rolling$rmse,
  dlwti_arma_rolling$rmse,
  dlwti_arma_manual_rolling$rmse,
  arima_res$rmse,
  armax_res$rmse,
  persist_result$rmse,
  avg_result$rmse,
  esm_result$rmse,
  linreg_result$rmse
)

aic_values = c(
  AIC(dwti_arma),
  AIC(dlwti_arma),
  AIC(dlwti_arma_manual),
  AIC(arima_model),
  AIC(armax_model),
  NA,
  NA,
  NA,
  NA
)

results_table = data.frame(
  Model = model_names,
  Rolling_RMSE = rmse_values,
  AIC = aic_values
)

print(results_table)
```

Saving Data

```{r}
# Saving data

#write.table(usd_idx_train_ts, "savedData/usd_idx_train_ts_new.csv", sep = ",", row.names =
#write.table(usd_idx_holdout_ts, "savedData/usd_idx_holdout_ts_new.csv", sep = ",", row.nam

#write.table(ind_prod_train_ts, "savedData/ind_prod_train_ts_new.csv", sep = ",", row.name
#write.table(ind_prod_holdout_ts, "savedData/ind_prod_holdout_ts_new.csv", sep = ",", row.
#write.table(crude_oil_ts, "savedData/crude_oil_ts_new.csv", sep = ",", row.names = FALSE)

#write.table(opec_prod_train_ts, "savedData/opec_prod_train_ts_new.csv", sep = ",", row.nam
#write.table(opec_prod_holdout_ts, "savedData/opec_prod_holdout_ts_new.csv", sep = ",", rou

#write.table(d_usd_ts, "savedData/d_usd_ts_new.csv", sep = ",", row.names = FALSE)
#write.table(dl_usd_idx_train_ts, "savedData/dl_usd_idx_train_ts_new.csv", sep = ",", row.
#write.table(d_ind_ts, "savedData/d_ind_ts_new.csv", sep = ",", row.names = FALSE)
#write.table(dl_ind_train_ts, "savedData/dl_ind_train_ts_new.csv", sep = ",", row.names =
```

```
#write.table(d_opec_ts, "savedData/d_opec_ts_new.csv", sep = ",", row.names = FALSE)
#write.table(dl_opec_idx_train_ts, "savedData/dl_opec_idx_train_ts_new.csv", sep = ",", rou
```